



UNIVERSIDAD
DE SANTIAGO
DE CHILE

Clasificación automática de líneas espectrales de emisión y absorción mediante modelos de deep learning

Benjamín Moya

Franco Pinto

Camila San Martín

Felipe Silva

Índice



Objetivos del proyecto



Descripción de la base de datos



Esquema metodológico



Implementación, entrenamiento
y validación



Conclusiones y mejoras



Referencias

Objetivo general

Desarrollar un modelo basado en redes neuronales convolucionales capaz de detectar líneas de emisión y absorción en espectros estelares, con un rendimiento superior al de los algoritmos actuales utilizados para esta clasificación.

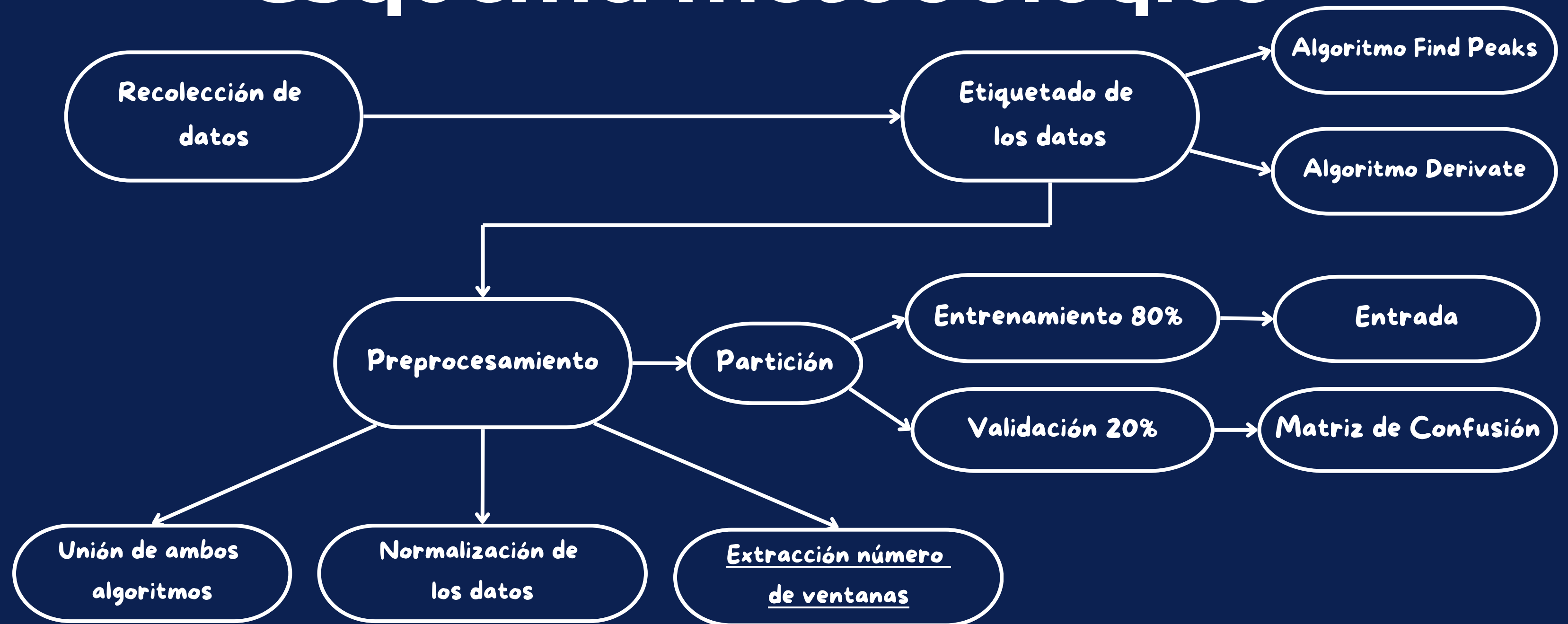
Objetivos específicos

- Identificar la información disponible sobre las líneas de emisión y absorción.
- Recopilar una base de datos representativa que incluya una variedad de espectros estelares.
- Desarrollar un sistema automatizado para el etiquetado de las líneas de emisión y absorción en los espectros, utilizando dos algoritmos y evaluar su efectividad mediante el análisis visual de algunos gráficos generados.
- Diseñar un modelo de deep learning adecuado para la detección y clasificación precisa de las líneas espectrales.
- Comparar el rendimiento de los modelos propuestos con los algoritmos utilizados.

Descripción de la base de datos

- ID: Identificador del espectro que se está analizando.
- Wavelength: Longitud de onda en la que se mide la intensidad del espectro, expresada en angstroms (Å). Representa las diferentes longitudes de onda de la luz analizada.
- Flux: Intensidad de radiación recibida por unidad de tiempo, área y longitud de onda ($\text{erg/s/cm}^2/\text{Å}$), que indica la energía detectada en cada longitud de onda.
- Em_peak y Abs_peak: Indicador de líneas de emisión y absorción detectadas buscando picos y valles en el flujo suavizado, filtrados según su diferencia con el continuo local.
- Em_deriv y Abs_deriv: Índices de líneas de emisión y absorción detectadas analizando cambios de signo en la derivada del flujo suavizado, aplicando umbrales para confirmar su relevancia.

Esquema metodológico



Arquitecturas usadas

Característica	CNN ID	LSTM	GRU
Tipo de datos	Datos secuenciales	Secuencias (orden importa)	Secuencias (orden importa)
¿Qué detecta mejor?	Patrones locales	Relaciones de largo plazo en la secuencia	Relaciones medias a largas
Velocidad / Complejidad	Muy rápida / Baja	Más lenta / Alta	Más rápida que LSTM / Media
Parámetros	Muchos	Menos que CNN	Menos que LSTM
Ventaja principal	Aprende formas locales sin necesidad de contexto	Aprende dependencias a largo plazo	Buena capacidad secuencial con menor complejidad

Implementación CNN 1D

```
model = Sequential([
    Conv1D(256, kernel_size=5, padding='same', kernel_initializer=HeNormal(), kernel_regularizer=l2(1e-4),
        input_shape=(X_train.shape[1], X_train.shape[2])),
    BatchNormalization(),
    LeakyReLU(),
    MaxPooling1D(pool_size=2),

    Conv1D(512, kernel_size=3, padding='same', kernel_initializer=HeNormal(), kernel_regularizer=l2(1e-4)),
    BatchNormalization(),
    LeakyReLU(),
    MaxPooling1D(pool_size=2),

    Conv1D(1024, kernel_size=3, padding='same', kernel_initializer=HeNormal(), kernel_regularizer=l2(1e-4)),
    BatchNormalization(),
    LeakyReLU(),

    GlobalAveragePooling1D(),

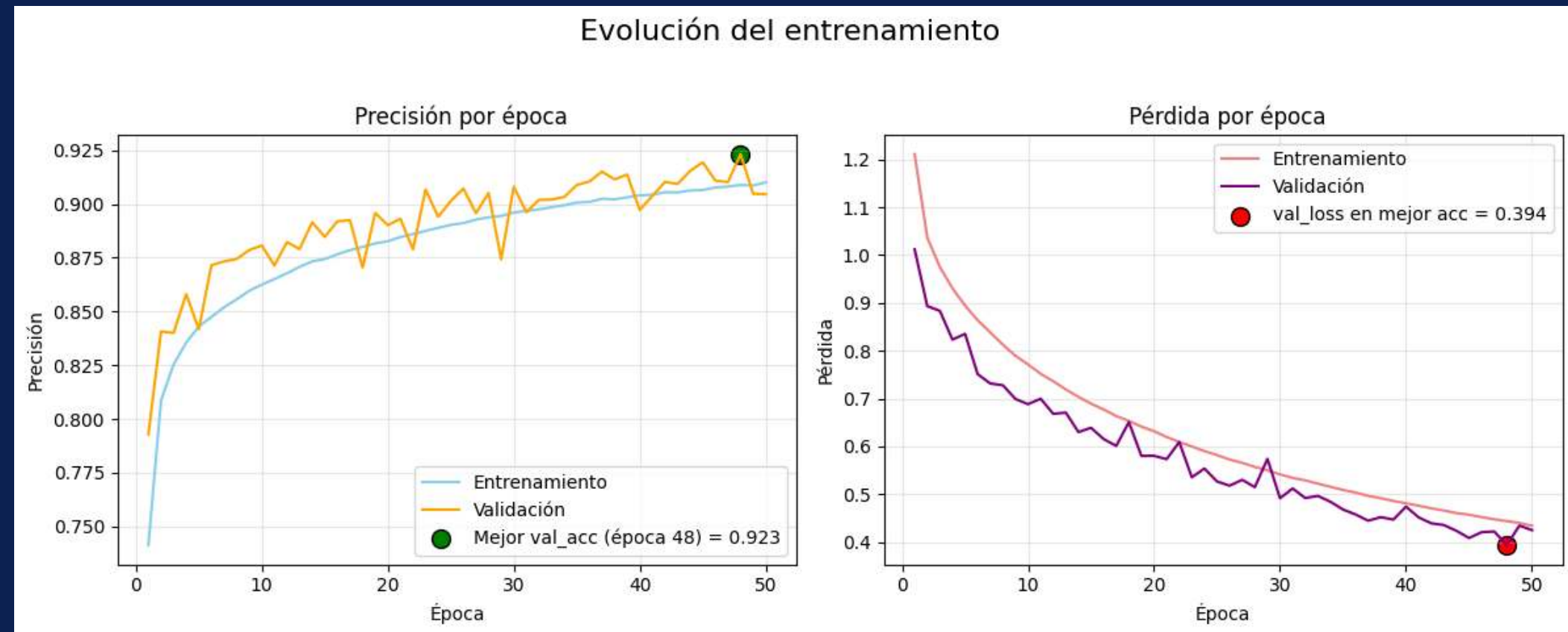
    Dense(512, activation='relu', kernel_regularizer=l2(1e-4)),
    Dropout(0.5),
    Dense(1024, activation='relu', kernel_regularizer=l2(1e-4)),
    Dropout(0.5),
    Dense(256, activation='relu', kernel_regularizer=l2(1e-4)),
    Dropout(0.3),
    Dense(128, activation='relu', kernel_regularizer=l2(1e-4)),
    Dropout(0.3),
    Dense(3, activation='softmax')
])

model.compile(optimizer=Adam(learning_rate=3e-5), loss='categorical_crossentropy', metrics=['accuracy'])
```

Layer (type)	Output Shape	Param #
=====		
conv1d_6 (Conv1D)	(None, 27, 256)	2816
batch_normalization_6 (Batch Normalization)	(None, 27, 256)	1024
leaky_re_lu_6 (LeakyReLU)	(None, 27, 256)	0
max_pooling1d (MaxPooling1D)	(None, 13, 256)	0
conv1d_7 (Conv1D)	(None, 13, 512)	393728
batch_normalization_7 (Batch Normalization)	(None, 13, 512)	2048
leaky_re_lu_7 (LeakyReLU)	(None, 13, 512)	0
max_pooling1d_1 (MaxPooling1D)	(None, 6, 512)	0
conv1d_8 (Conv1D)	(None, 6, 1024)	1573888
...		
Total params: 3,323,395		
Trainable params: 3,319,811		
Non-trainable params: 3,584		

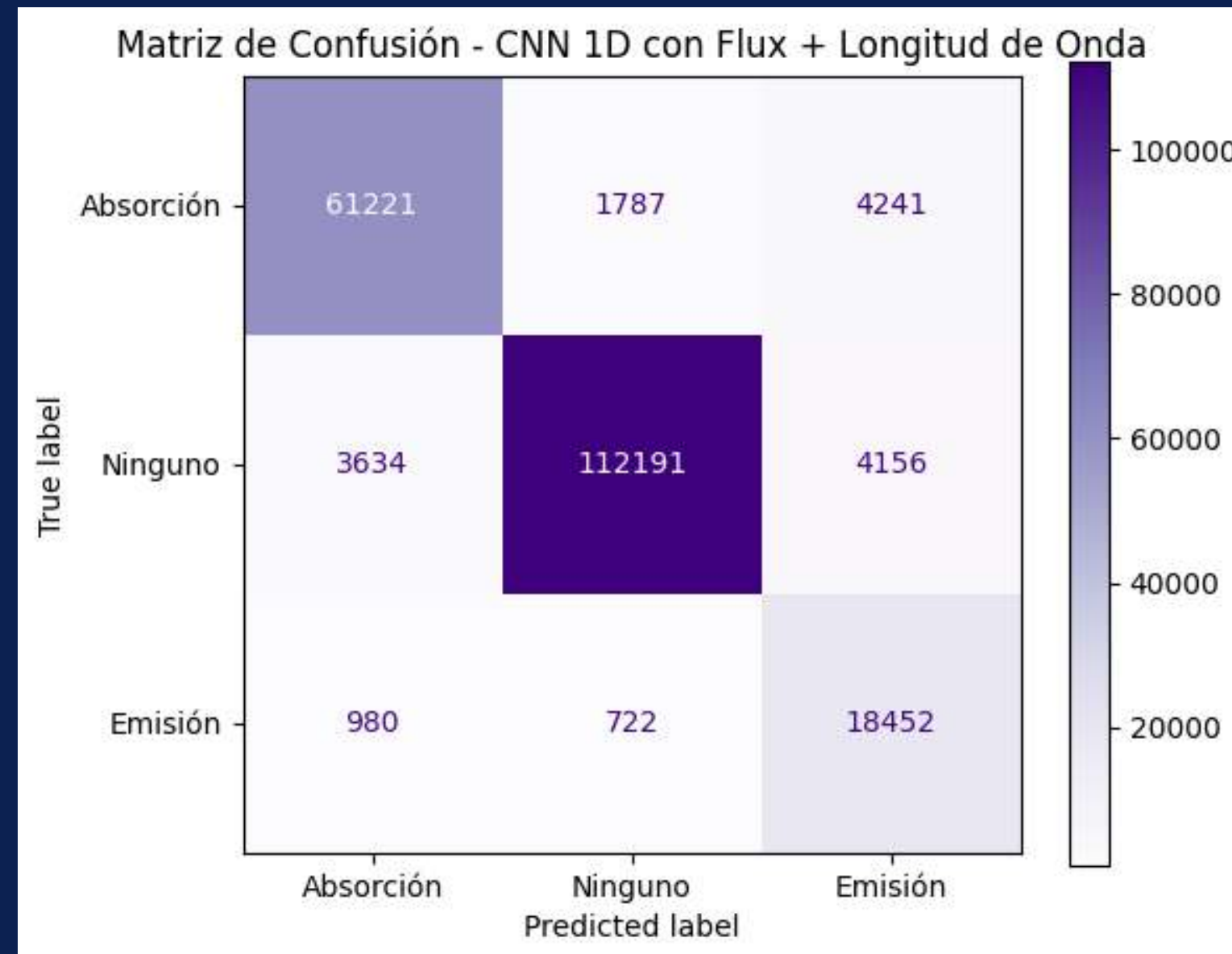
Entrenamiento CNN 1D

- Épocas: 50
- GPU: Sí
- Plataforma: Visual Studio Code
- Tiempo: 62m 9.9s
- Código



Validación CNN 1D

- Precisión: 92.5%



Implementación LSTM

```
modelo_lstm = Sequential([
    Bidirectional(LSTM(128, return_sequences=True),
        input_shape=(X_train.shape[1], X_train.shape[2])),
    Dropout(0.3),
    Bidirectional(LSTM(64)),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dropout(0.2),
    Dense(3, activation='softmax')
])

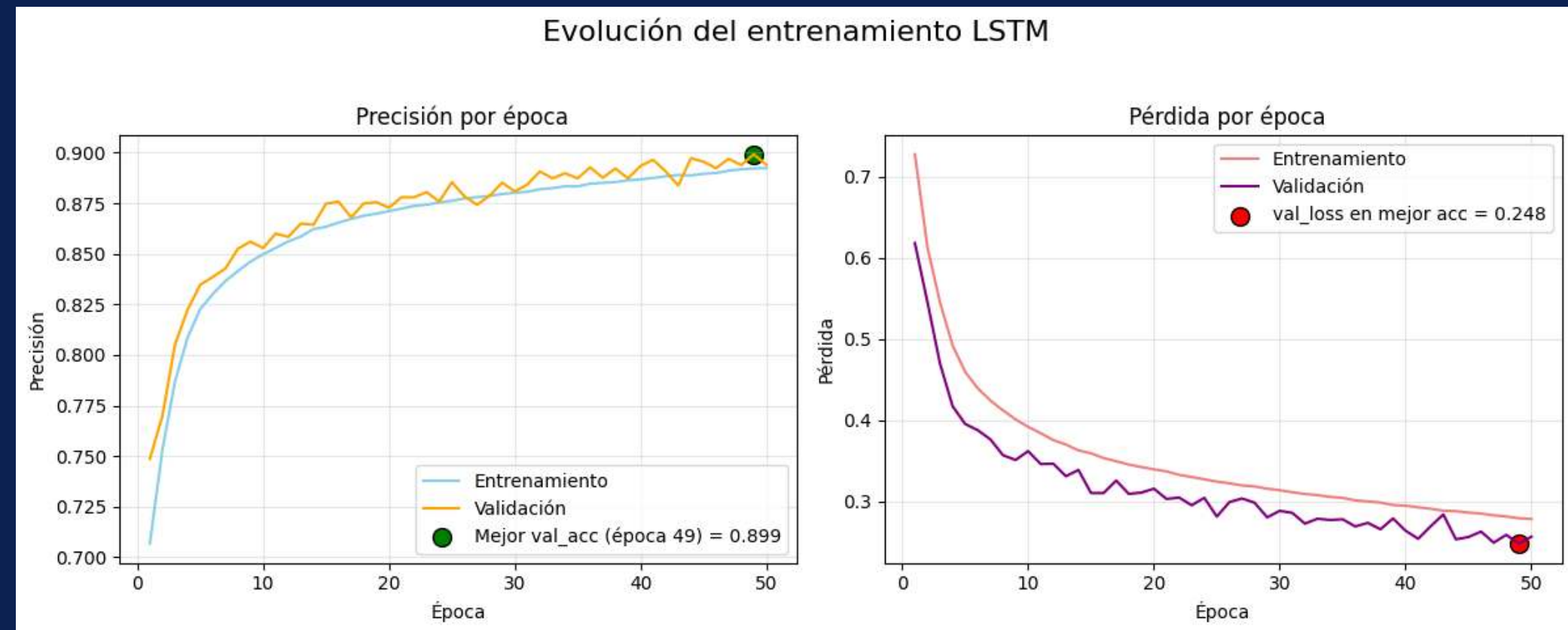
modelo_lstm.compile(optimizer=Adam(1e-4),
    loss='categorical_crossentropy',
    metrics=['accuracy'])
```

Layer (type)	Output Shape	Param #
bidirectional_3 (Bidirectional)	(None, 27, 256)	134144
dropout_10 (Dropout)	(None, 27, 256)	0
bidirectional_4 (Bidirectional)	(None, 128)	164352
dropout_11 (Dropout)	(None, 128)	0
dense_14 (Dense)	(None, 64)	8256
dropout_12 (Dropout)	(None, 64)	0
dense_15 (Dense)	(None, 3)	195

=====
Total params: 306,947
Trainable params: 306,947
Non-trainable params: 0

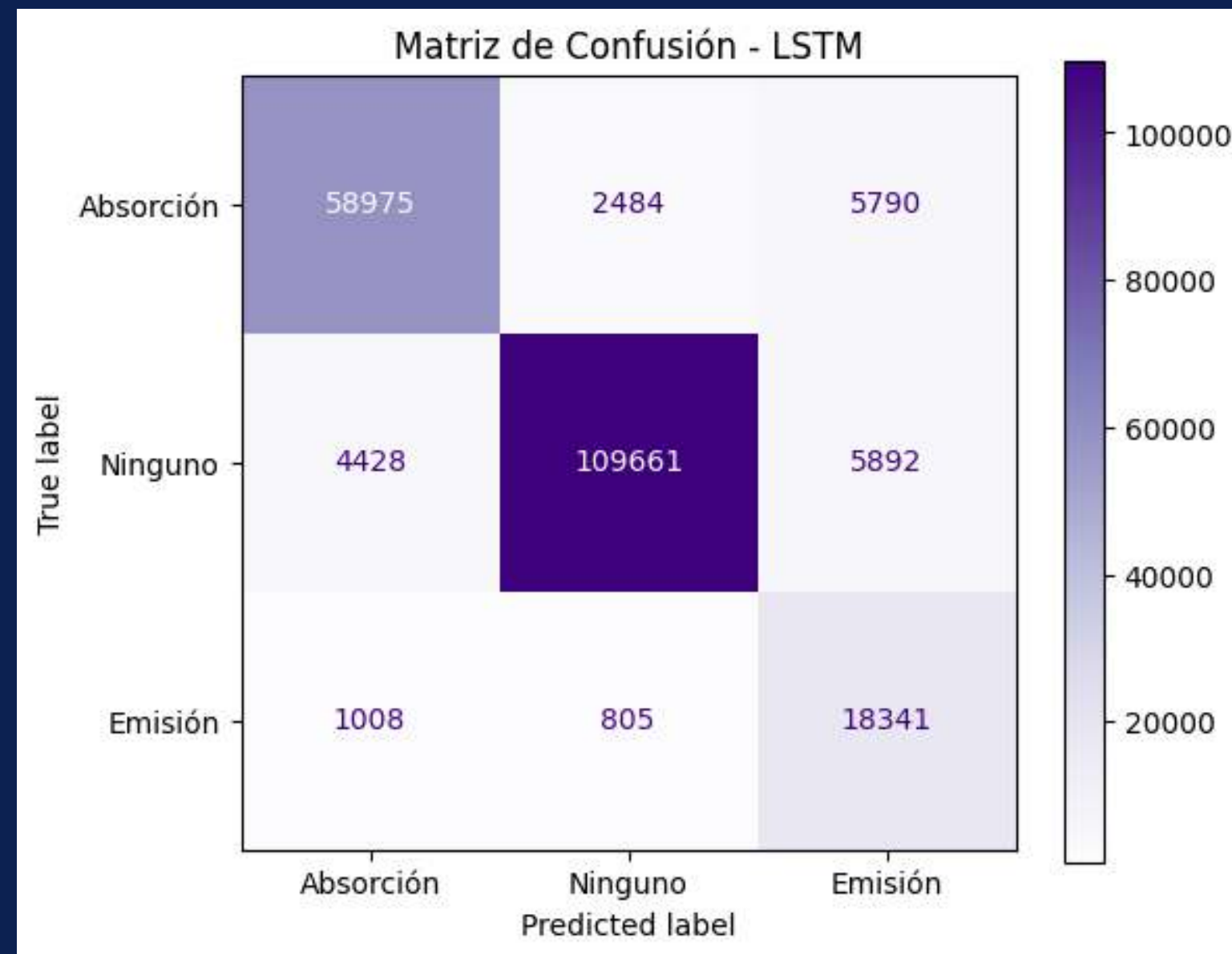
Entrenamiento LSTM

- Épocas: 50
- GPU: Sí
- Plataforma: Visual Studio Code
- Tiempo: 91m 12.9s
- Código



Validación LSTM

- Precisión: 90.1%



Implementación CNN 1D + LSTM

```
modelo_robusto = Sequential([
    # --- Extracción local de patrones con conv ---
    Conv1D(128, kernel_size=3, padding='same', kernel_regularizer=l2(1e-4),
        input_shape=(X_train.shape[1], X_train.shape[2])),
    BatchNormalization(),
    LeakyReLU(),

    Conv1D(256, kernel_size=3, padding='same', kernel_regularizer=l2(1e-4)),
    BatchNormalization(),
    LeakyReLU(),

    # --- LSTM bidireccional para patrones secuenciales ---
    Bidirectional(LSTM(128, return_sequences=False, dropout=0.2)),

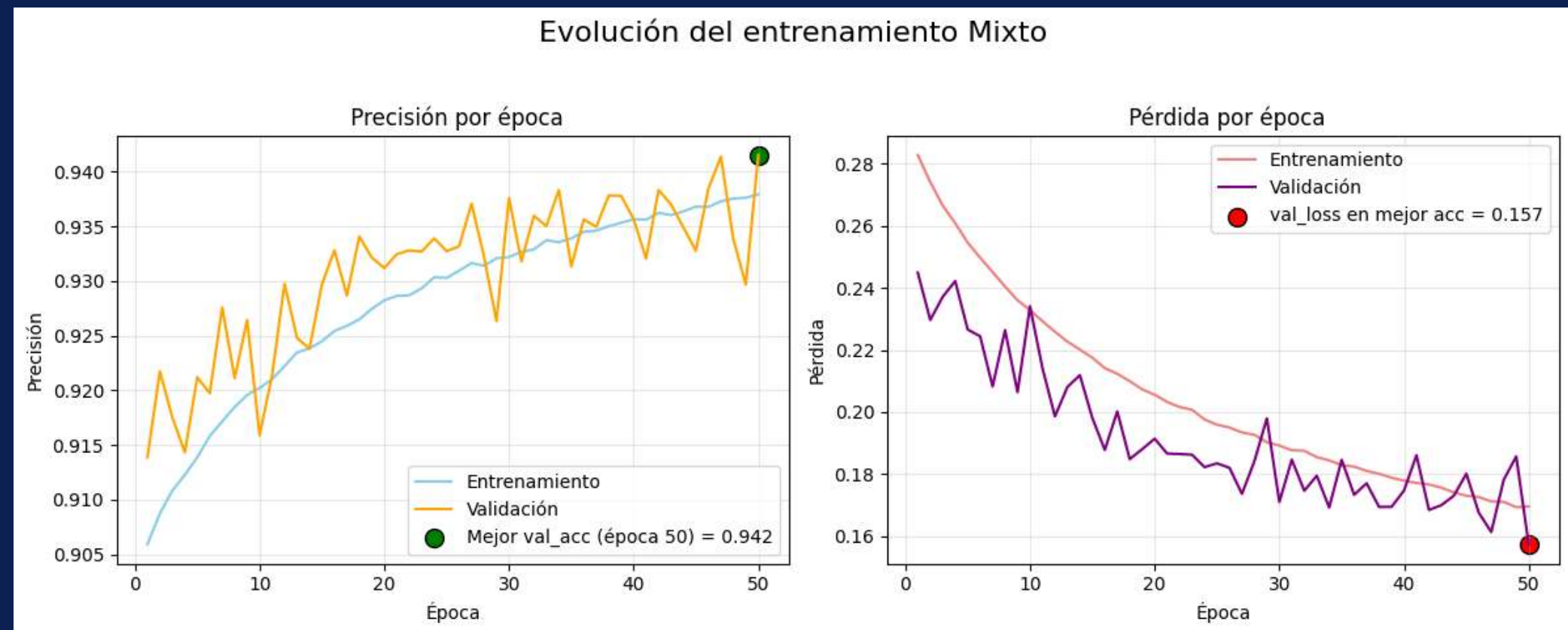
    # --- Denso profundo ---
    Dense(256, activation='relu', kernel_regularizer=l2(1e-4)),
    Dropout(0.4),
    Dense(128, activation='relu', kernel_regularizer=l2(1e-4)),
    Dropout(0.3),

    # --- Salida ---
    Dense(3, activation='softmax')
])
modelo_robusto.summary()
modelo_robusto.compile(
    optimizer=Adam(learning_rate=1e-4),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

Layer (type)	Output Shape	Param #
=====		
conv1d_4 (Conv1D)	(None, 27, 128)	896
batch_normalization_4 (Batch Normalization)	(None, 27, 128)	512
leaky_re_lu_4 (LeakyReLU)	(None, 27, 128)	0
conv1d_5 (Conv1D)	(None, 27, 256)	98560
batch_normalization_5 (Batch Normalization)	(None, 27, 256)	1024
leaky_re_lu_5 (LeakyReLU)	(None, 27, 256)	0
bidirectional_2 (Bidirectional)	(None, 256)	394240
dense_6 (Dense)	(None, 256)	65792
dropout_4 (Dropout)	(None, 256)	0
...		
Total params: 594,307		
Trainable params: 593,539		
Non-trainable params: 768		

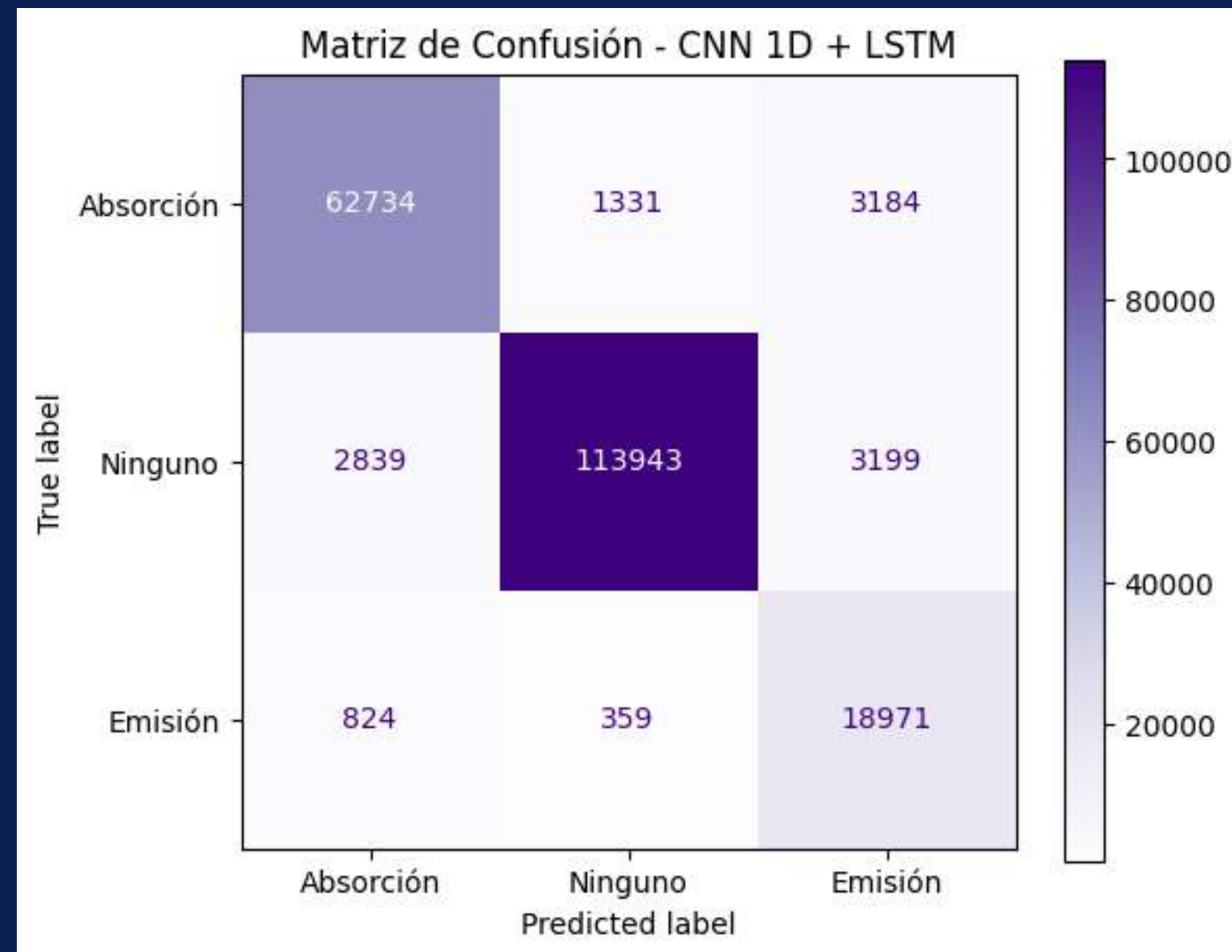
Entrenamiento CNN 1D + LSTM

- Épocas: 50
- GPU: Sí
- Plataforma: Visual Studio Code
- Tiempo: 75m 0.5s
- Código



Validación CNN 1D + LSTM

- Precisión: 94.3%



Implementación CNN 1D + GRU

```
modelo_cnn_gru = Sequential([
    Conv1D(128, kernel_size=3, padding='same',
          input_shape=(X_train.shape[1], X_train.shape[2])),
    BatchNormalization(),
    LeakyReLU(),

    Conv1D(256, kernel_size=3, padding='same'),
    BatchNormalization(),
    LeakyReLU(),

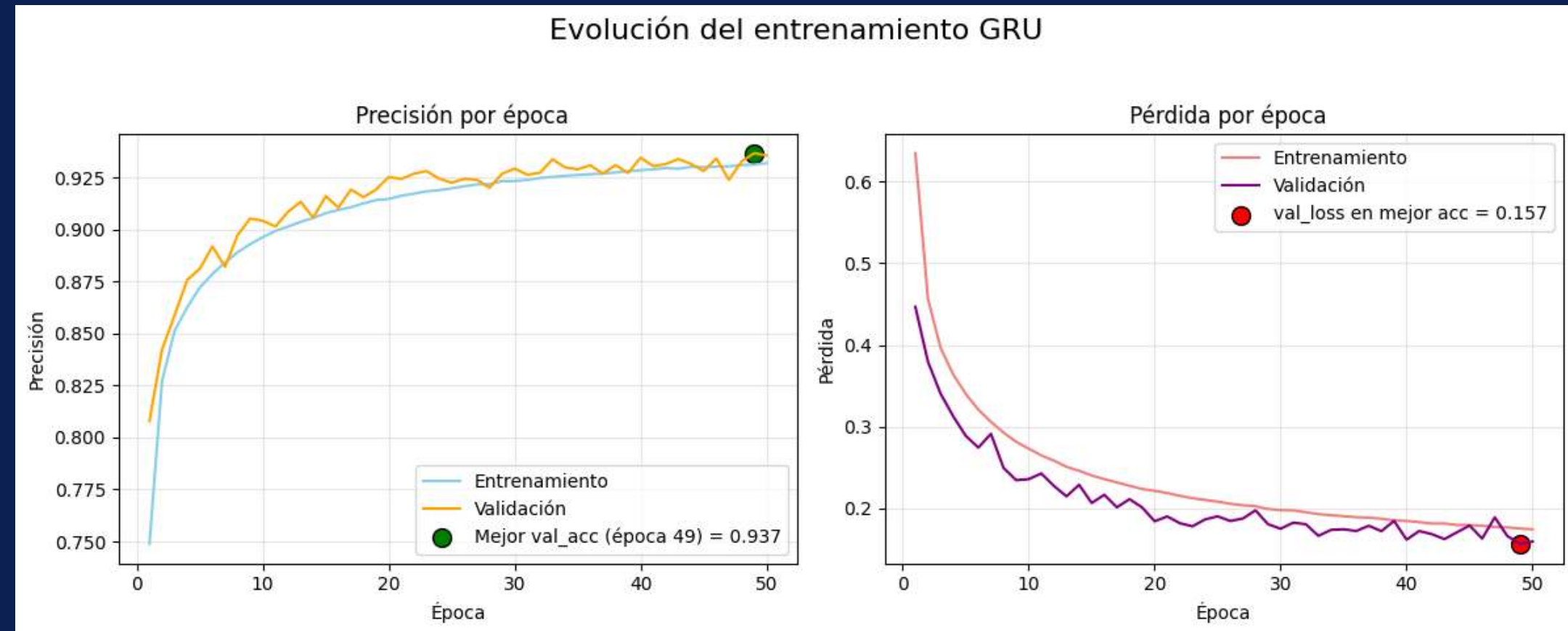
    GRU(128, return_sequences=False, dropout=0.2),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(3, activation='softmax')
])

modelo_cnn_gru.compile(optimizer=Adam(1e-4),
                       loss='categorical_crossentropy',
                       metrics=['accuracy'])
```

Layer (type)	Output Shape	Param #
conv1d_9 (Conv1D)	(None, 27, 128)	896
batch_normalization_9 (Batch Normalization)	(None, 27, 128)	512
leaky_re_lu_9 (LeakyReLU)	(None, 27, 128)	0
conv1d_10 (Conv1D)	(None, 27, 256)	98560
batch_normalization_10 (Batch Normalization)	(None, 27, 256)	1024
leaky_re_lu_10 (LeakyReLU)	(None, 27, 256)	0
gru (GRU)	(None, 128)	148224
dense_16 (Dense)	(None, 128)	16512
dropout_13 (Dropout)	(None, 128)	0
dense_17 (Dense)	(None, 3)	387
...		
Total params: 266,115		
Trainable params: 265,347		
Non-trainable params: 768		

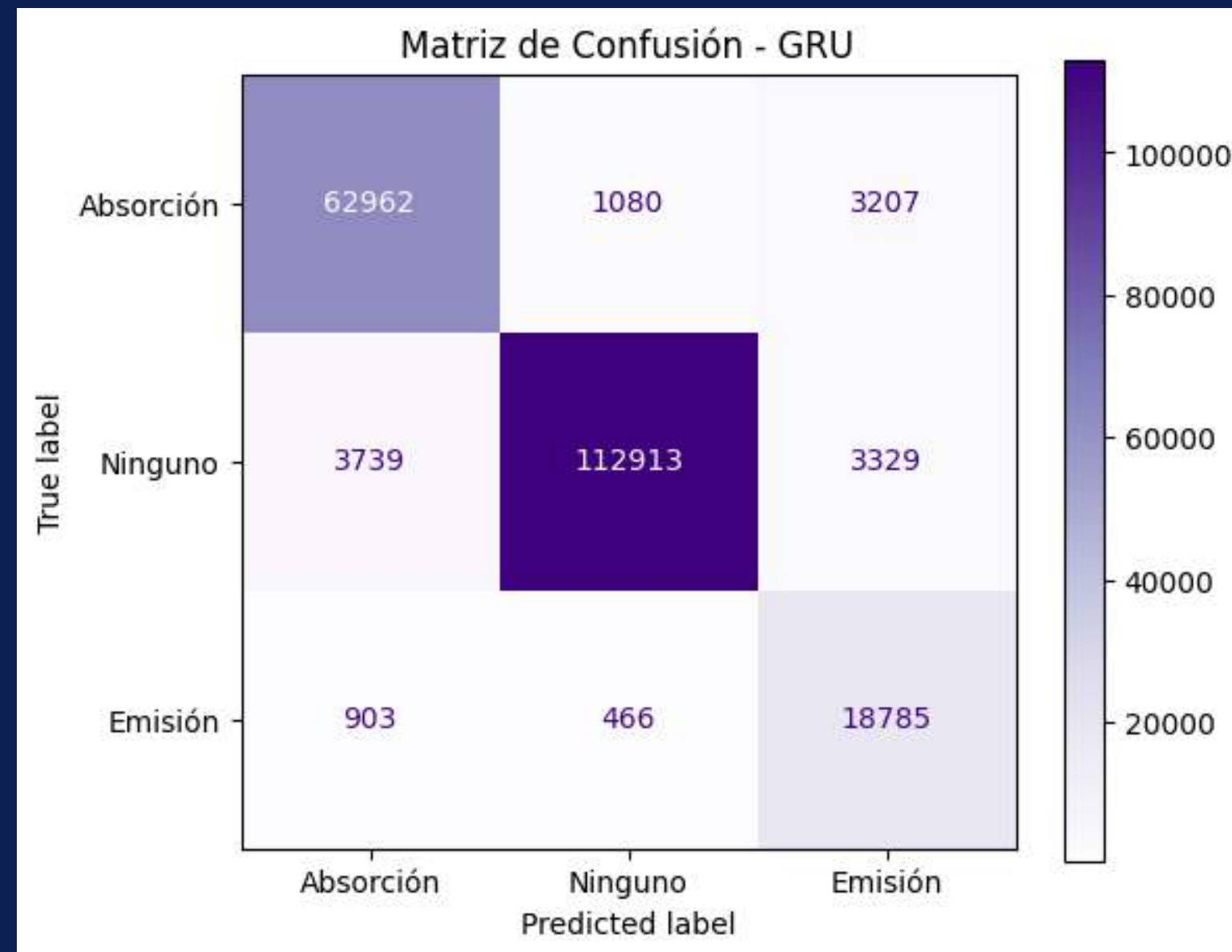
Entrenamiento CNN 1D + GRU

- Épocas: 50
- GPU: Sí
- Plataforma: Visual Studio Code
- Tiempo: 46m 23.6s
- Código



Validación CNN 1D + GRU

- Precisión: 93.8%



Resumen métricas

Modelo	Precisión	Tiempo	Parámetros
CNN ID	92.5%	62m 9.9s	3 319 811
LSTM	90.1%	91m 12.9s	306 947
CNN ID + LSTM	94.3%	75m 0.5s	593 539
CNN ID + GRU	93.8%	46m 23.6s	265 347

Conclusiones

- Los modelos desarrollados y entrenados lograron un buen rendimiento en la clasificación por ventanas, mostrando una capacidad efectiva para discriminar entre regiones con o sin presencia de líneas espectrales.
- El enfoque adoptado se centró en la clasificación de segmentos del espectro, más que en la identificación puntual de la ubicación exacta de las líneas de emisión o absorción. Esto representa una desviación parcial respecto al objetivo inicial del proyecto.
- Si bien no fue posible realizar una comparación con los algoritmos automáticos de detección de líneas, el modelo propuesto representa un aporte complementario, que podría integrarse a sistemas existentes para mejorar su desempeño.

Mejoras

- Refinar la estrategia de etiquetado por ventana.
- Entrenar el modelo con una base más informativa.
- Modificar el enfoque del modelo o hacer un segundo modelo para que en lugar de clasificar ventanas del espectro, sea capaz de identificar la ubicación exacta de las líneas de emisión y absorción dentro del espectro completo.
- Comprobar efectividad del modelo con una base clasificada por expertos.

Referencias

- Sloan Digital Sky Survey. (s.f.). MaNGA Survey - MaStar. <https://www.sdss4.org/surveys/mastar/>
- Sloan Digital Sky Survey. (s.f.). MaNGA survey. <https://www.sdss4.org/surveys/manga/>
- Specutils Developers. (2025). Line/Spectrum Fitting. Specutils Documentation. <https://specutils.readthedocs.io/en/stable/fitting.html>
- Sloan Digital Sky Survey. (2021). MaNGA Spectro data: Mastar v3_1_1, v1_7_7. https://data.sdss.org/sas/dr17/manga/spectro/mastar/v3_1_1/v1_7_7/
- Plaza, F. (2025). Deep Learning 04: RNN [Material de curso]. Recuperado el 10 de julio de 2025, de https://franplaza.github.io/courses/deep_learning/Deep_Learning_04_RNN.html
- MathWorks. (s.f.). Introducción a la memoria a corto-largo plazo (LSTM). Material de referencia. Recuperado el 10 de julio de 2025, de la página de MathWorks: la.mathworks.com/discovery/lstm.html
- Nama, A. (4 de mayo de 2023). Understanding Gated Recurrent Unit (GRU) in Deep Learning. Medium. Recuperado el 10 de julio de 2025, de <https://medium.com/@anishnama20/understanding-gated-recurrent-unit-gru-in-deep-learning-2e54923f3e2>



**¡muchas
gracias!**

CNN 1D

```
checkpoint = ModelCheckpoint(  
    filepath='mejor_modelo.h5',  
    monitor='val_accuracy',  
    save_best_only=True,  
    save_weights_only=False,  
    verbose=1  
)  
  
history = model.fit(  
    X_train, y_train,  
    epochs=50,  
    batch_size=256,  
    validation_split=0.2,  
    class_weight=class_weight_dict,  
    callbacks=[checkpoint]  
)
```

[Return](#)

LSTM

```
checkpoint = ModelCheckpoint(  
    filepath='mejor_modelo_LSTM.h5',  
    monitor='val_accuracy',  
    save_best_only=True,  
    save_weights_only=False,  
    verbose=1  
)  
history_LSTM = modelo_lstm.fit(  
    X_train, y_train,  
    epochs=50,  
    batch_size=256,  
    validation_split=0.2,  
    class_weight=class_weight_dict,  
    callbacks=[checkpoint]  
)
```

[Return](#)

CNN 1D + LSTM

```
checkpoint = ModelCheckpoint(  
    filepath='mejor_modelo_R.h5',  
    monitor='val_accuracy',  
    save_best_only=True,  
    save_weights_only=False,  
    verbose=1  
)  
  
history_R = modelo_robusto.fit(  
    X_train, y_train,  
    epochs=50,  
    batch_size=256,  
    validation_split=0.2,  
    class_weight=class_weight_dict,  
    callbacks=[checkpoint]  
)
```

[Return](#)

CNN 1D + GRU

```
from tensorflow.keras.optimizers import Adam
checkpoint = ModelCheckpoint(
    filepath='mejor_modelo_GRU.h5',
    monitor='val_accuracy',
    save_best_only=True,
    save_weights_only=False,
    verbose=1
)
history_gru = modelo_cnn_gru.fit(
    X_train, y_train,
    epochs=10,
    batch_size=256,
    validation_split=0.2,
    class_weight=class_weight_dict,
    callbacks=[checkpoint])
```

[Return](#)

Extracción de ventana

Algoritmo

```
x = []
y = []

for _, grupo in df_pd.groupby('ID'):
    flujo = grupo['flux'].values
    onda = grupo['wave'].values
    em = grupo['em_peak'].values
    absb = grupo['abs_peak'].values

    # Estandarización por espectro (puntaje Z)
    flujo_mean = flujo.mean()
    flujo_std = flujo.std()
    flujo = (flujo - flujo_mean) / (flujo_std + 1e-8)

    onda_mean = onda.mean()
    onda_std = onda.std()
    onda = (onda - onda_mean) / (onda_std + 1e-8)

    for i in range(0, len(flujo) - 27, 27):
        ventana_flujo = flujo[i:i+27]
        ventana_onda = onda[i:i+27]
        etiqueta = 0
        if em[i:i+27].sum() > 0:
            etiqueta = 1
        elif absb[i:i+27].sum() > 0:
            etiqueta = -1
        ventana = np.stack([ventana_flujo, ventana_onda], axis=-1)
        x.append(ventana)
        y.append(etiqueta)
```

X[0]

```
0.64176323, -0.43992838],
0.64279557, -0.43926353],
0.64510326, -0.43859853],
0.65040807, -0.43793338],
0.65193184, -0.43726807],
0.64760684, -0.4366026 ],
0.63705365, -0.43593699],
0.63507643, -0.43527122],
0.63470111, -0.4346053 ],
0.62094496, -0.43393922],
0.60897964, -0.433273 ],
0.69998348, -0.43260661],
0.8613018 , -0.43194008],
0.91871724, -0.43127339],
0.87288397, -0.43060655],
0.85444971, -0.42993955],
0.86361415, -0.4292724 ],
0.861517 , -0.4286051 ],
0.75715482, -0.42793764],
0.57954015, -0.42727003],
0.51215768, -0.42660227],
0.55242273, -0.42593435],
0.57410889, -0.42526628],
0.57793172, -0.42459805],
0.58800261, -0.42392967],
0.60426642, -0.42326114],
0.61725886, -0.42259245]]
```

Y[0]

1

Total de ventanas

Absorción: 336247 ventanas
Nada: 600000 ventanas
Emisión: 100768 ventanas

Return